

Documento de Requisitos

Trabalho Final — Problema do Bar dos Filósofos em Python

1. Identificação do Projeto

Nome do projeto: Simulação do Problema do Bar dos Filósofos

Disciplina: Sistemas Distribuídos / Programação Concorrente

Linguagem escolhida: Python

Tipo de trabalho: Trabalho prático em grupo

Tamanho do grupo: mínimo de 2 e máximo de 4 alunos

Objetivo geral: Implementar e comparar soluções concorrentes para o Problema do Bar dos Filósofos, garantindo exclusão mútua no acesso às garrafas, evitando deadlock e avaliando possíveis situações de starvation.

2. Contextualização do Problema

O Problema do Bar dos Filósofos é uma generalização do clássico Problema do Jantar dos Filósofos. No jantar dos filósofos, os processos são organizados em uma estrutura circular e cada filósofo precisa dos dois garfos vizinhos para comer.

No Bar dos Filósofos, a estrutura deixa de ser obrigatoriamente circular e passa a ser representada por um grafo genérico. Cada vértice do grafo representa um filósofo, enquanto cada aresta representa uma garrafa compartilhada entre dois filósofos. Assim, os filósofos podem precisar de diferentes conjuntos de garrafas para beber.

O problema exige o controle correto de recursos compartilhados em um ambiente concorrente. Como vários filósofos podem tentar acessar as mesmas garrafas ao mesmo tempo, é necessário usar mecanismos de sincronização para garantir exclusão mútua, evitar deadlock e reduzir a possibilidade de starvation.

3. Objetivos do Sistema

O sistema deverá simular o comportamento de filósofos em um bar, utilizando threads para representar a execução concorrente de cada filósofo.

Os principais objetivos são:

1. Representar os filósofos como processos ou threads independentes.
 2. Representar as garrafas como recursos compartilhados entre filósofos.
 3. Permitir que o grafo do problema seja lido a partir de um arquivo `.txt`.
 4. Implementar pelo menos duas soluções para controle de concorrência.
 5. Garantir que uma garrafa não seja usada por mais de um filósofo ao mesmo tempo.
 6. Evitar deadlock durante toda a execução.
 7. Coletar tempos de execução e espera de cada filósofo.
 8. Comparar as soluções implementadas com base nos tempos obtidos.
 9. Avaliar se houve ou não starvation.
 10. Apresentar os resultados finais de forma clara no terminal ou em arquivo de saída.
-

4. Linguagem e Tecnologias Escolhidas

4.1 Linguagem de Programação

A linguagem escolhida para a implementação será **Python**.

4.2 Justificativa da Escolha

Python foi escolhido por possuir bibliotecas nativas que facilitam a implementação de programação concorrente, como:

- `threading`: criação e controle de threads;
- `Lock`: controle de exclusão mútua;
- `Semaphore`: controle de acesso limitado;
- `time`: medição de tempo e pausas na execução;
- `random`: sorteio de tempos e quantidade de garrafas;
- `argparse`: leitura de argumentos pela linha de comando.

A linguagem também facilita a leitura de arquivos, a manipulação de listas e matrizes, além de permitir uma implementação mais clara para análise e apresentação dos resultados.

5. Escopo do Projeto

5.1 O que o sistema deve fazer

O sistema deve:

- Ler uma matriz de adjacência a partir de um arquivo `.txt`;
- Criar automaticamente os filósofos com base no número de vértices do grafo;
- Criar automaticamente as garrafas com base nas arestas do grafo;
- Executar uma simulação concorrente;

- Fazer cada filósofo alternar entre os estados tranquilo, com sede e bebendo;
- Fazer cada filósofo sortear as garrafas que deseja utilizar em cada rodada;
- Controlar o acesso às garrafas com exclusão mútua;
- Implementar pelo menos duas soluções diferentes;
- Registrar os tempos de cada filósofo em cada estado;
- Calcular o tempo médio de espera de cada filósofo;
- Exibir os resultados finais da simulação;
- Comparar as soluções implementadas.

5.2 O que o sistema não precisa fazer

O sistema não precisa:

- Ter interface gráfica obrigatória;
 - Usar banco de dados;
 - Simular uma rede real entre computadores diferentes;
 - Usar múltiplos processos distribuídos reais;
 - Implementar todas as soluções sugeridas no enunciado;
 - Fazer animação visual do grafo;
 - Alterar o grafo durante a execução.
-

6. Representação do Problema

6.1 Filósofos

Cada filósofo será representado por uma thread.

Cada filósofo terá:

- um identificador numérico;
- uma lista de garrafas adjacentes;
- um estado atual;
- contadores de tempo em cada estado;
- contador de quantas vezes já bebeu;
- controle de início e fim de cada estado.

6.2 Garrafas

Cada garrafa será representada por uma aresta do grafo.

Como uma garrafa é compartilhada entre dois filósofos, ela deverá possuir um mecanismo de exclusão mútua, como um **Lock**.

Exemplo:

```
garrafa = threading.Lock()
```

Cada garrafa deve ter um identificador único, permitindo que as soluções baseadas em ordenação consigam sempre pegar os recursos na mesma ordem.

6.3 Grafo

O grafo será representado por uma matriz de adjacência.

Na matriz:

- 1 indica que existe uma aresta entre dois filósofos;
- 0 indica que não existe aresta entre eles.

Exemplo de matriz:

```
0, 1, 0, 0, 1
1, 0, 1, 0, 0
0, 1, 0, 1, 0
0, 0, 1, 0, 1
1, 0, 0, 1, 0
```

Essa matriz representa o caso clássico do Jantar dos Filósofos com 5 nós.

7. Estados dos Filósofos

Cada filósofo deverá possuir três estados principais, seguindo a sequência:

tranquilo → com sede → bebendo → tranquilo

7.1 Estado tranquilo

O filósofo está descansando e não está tentando acessar garrafas.

O tempo nesse estado deve ser sorteado aleatoriamente entre 0 e o número de garrafas adjacentes ao filósofo.

Exemplo:

```
tempo_tranquilo = random.uniform(0, grau_do_vertice)
```

7.2 Estado com sede

O filósofo deseja beber e tenta adquirir um subconjunto de garrafas adjacentes.

Esse tempo corresponde ao período de espera até conseguir todas as garrafas necessárias.

Esse tempo é muito importante para a análise de starvation.

7.3 Estado bebendo

O filósofo conseguiu adquirir todas as garrafas necessárias e pode beber.

O tempo de bebida deve ser de 1 segundo.

Após beber, o filósofo libera todas as garrafas adquiridas e retorna ao estado tranquilo.

8. Requisitos Funcionais

RF01 — Leitura da matriz de adjacência

O sistema deve ser capaz de ler um arquivo `.txt` contendo a matriz de adjacência do grafo.

O arquivo poderá conter valores separados por vírgula, como:

```
0, 1, 0, 0, 1
1, 0, 1, 0, 0
0, 1, 0, 1, 0
0, 0, 1, 0, 1
1, 0, 0, 1, 0
```

O programa deve interpretar cada linha como uma linha da matriz.

RF02 — Validação da matriz

O sistema deve verificar se:

- a matriz é quadrada;
 - os valores são apenas 0 ou 1;
 - a diagonal principal possui apenas 0;
 - a matriz é simétrica, pois o grafo é não direcionado;
 - todos os filósofos possuem pelo menos uma garrafa adjacente.
-

RF03 — Criação automática dos filósofos

O sistema deve criar automaticamente um filósofo para cada vértice da matriz.

Se a matriz possuir 5 linhas, o sistema deve criar 5 filósofos.

Se a matriz possuir 12 linhas, o sistema deve criar 12 filósofos.

RF04 — Criação automática das garrafas

O sistema deve criar uma garrafa para cada aresta do grafo.

Como a matriz é simétrica, a garrafa entre os filósofos *i* e *j* deve ser criada apenas uma vez.

Exemplo:

Se $M[0][1] = 1$ e $M[1][0] = 1$, existe uma única garrafa compartilhada entre os filósofos 0 e 1.

RF05 — Representação das garrafas com exclusão mútua

Cada garrafa deve ser protegida por um mecanismo de sincronização.

Em Python, recomenda-se utilizar `threading.Lock`.

Assim, uma mesma garrafa não poderá ser usada por dois filósofos ao mesmo tempo.

RF06 — Implementação da classe Filósofo

O sistema deve possuir uma classe `Filosofo`.

Essa classe deve conter, no mínimo:

- identificador do filósofo;
 - lista de garrafas disponíveis para aquele filósofo;
 - estado atual;
 - quantidade de vezes que o filósofo bebeu;
 - tempos acumulados nos estados tranquilo, com sede e bebendo;
 - método principal de execução da thread;
 - método para escolher as garrafas desejadas;
 - método para tentar adquirir garrafas;
 - método para liberar garrafas.
-

RF07 — Sorteio das garrafas desejadas

Quando um filósofo entrar no estado com sede, ele deve sortear um subconjunto de suas garrafas adjacentes.

A quantidade de garrafas deve variar entre 2 e n , sendo n o número de garrafas adjacentes ao filósofo.

Caso algum filósofo possua apenas uma garrafa adjacente, o sistema deve tratar esse caso de forma segura, permitindo que ele sorteie uma garrafa ou ajustando a regra conforme a conectividade do grafo.

RF08 — Tempo no estado tranquilo

O tempo no estado tranquilo deve ser aleatório.

Esse tempo deve variar entre 0 e o número de arestas adjacentes ao filósofo.

RF09 — Tempo no estado com sede

O tempo no estado com sede deve ser contado desde o momento em que o filósofo começa a tentar adquirir as garrafas até o momento em que consegue todas elas.

Esse tempo deve ser registrado individualmente para cada filósofo.

RF10 — Tempo no estado bebendo

O tempo no estado bebendo deve ser fixo em 1 segundo.

Após esse período, o filósofo deve liberar todas as garrafas adquiridas.

RF11 — Quantidade de ciclos de bebida

O sistema deve controlar quantas vezes cada filósofo deve beber.

Para os casos 1 e 2, cada filósofo deve beber 6 vezes.

Para o caso 3, cada filósofo deve beber 3 vezes.

Essa quantidade deve ser configurável no programa, para facilitar testes.

RF12 — Implementação de pelo menos duas soluções

O sistema deve implementar, no mínimo, duas estratégias de controle de concorrência.

As soluções recomendadas para implementação em Python são:

1. Ordenação de recursos;
2. Solução do garçom.

Essas duas soluções são recomendadas por serem mais simples de implementar, explicar e comparar.

RF13 — Solução 1: Ordenação de recursos

Na solução por ordenação de recursos, cada garrafa recebe um identificador único.

Quando um filósofo precisar de várias garrafas, ele sempre deve tentar adquiri-las em ordem crescente de identificador.

Essa regra evita espera circular e, conseqüentemente, ajuda a evitar deadlock.

Exemplo:

Se um filósofo precisa das garrafas 8, 2 e 5, ele deve tentar pegar na ordem:

$2 \rightarrow 5 \rightarrow 8$

RF14 — Solução 2: Garçom

Na solução do garçom, deve existir uma entidade central que controla a permissão para os filósofos tentarem pegar garrafas.

O garçom deve limitar a quantidade de filósofos tentando beber ao mesmo tempo.

Uma forma simples de implementar essa solução é utilizar um [Semaphore](#).

Exemplo:

```
garcom = threading.Semaphore(N - 1)
```

Assim, no máximo $N - 1$ filósofos poderão tentar adquirir garrafas simultaneamente.

RF15 — Liberação correta das garrafas

Após beber, o filósofo deve liberar todas as garrafas adquiridas.

Essa liberação deve ocorrer mesmo se algum erro acontecer durante a tentativa de execução.

Recomenda-se utilizar blocos `try/finally` para garantir que os recursos sejam liberados corretamente.

RF16 — Registro dos tempos individuais

O sistema deve registrar, para cada filósofo:

- tempo total tranquilo;
 - tempo total com sede;
 - tempo total bebendo;
 - quantidade de vezes que bebeu;
 - tempo médio de espera;
 - tempo total de vida da thread.
-

RF17 — Cálculo do tempo médio de espera

O tempo médio de espera de cada filósofo deve ser calculado pela fórmula:

tempo médio de espera = tempo total com sede / quantidade de vezes que bebeu

Esse valor será utilizado para avaliar se algum filósofo esperou muito mais do que os demais.

RF18 — Cálculo do tempo total da simulação

O sistema deve medir o tempo total da simulação desde o início da execução das threads até o final da execução de todos os filósofos.

RF19 — Exibição dos resultados

Ao final da execução, o sistema deve exibir uma tabela com os resultados.

Exemplo de saída esperada:

Resultados da Simulação — Solução: Ordenação de Recursos

Filósofo | Tranquilo(s) | Com sede(s) | Bebendo(s) | Bebeu | Espera média(s)

	-----	-----	-----	-----	-----	-----
F0	10.25	2.31	6.00	6	0.38	
F1	9.87	3.10	6.00	6	0.52	
F2	11.40	2.95	6.00	6	0.49	

Tempo total da simulação: 23.45 segundos

RF20 — Comparação entre soluções

O sistema deve permitir comparar as soluções implementadas.

A comparação deve considerar:

- tempo total da simulação;
- tempo médio de espera de cada filósofo;
- maior tempo de espera;
- menor tempo de espera;
- diferença entre o maior e o menor tempo de espera;
- ocorrência ou não de deadlock;
- indícios de starvation.

RF21 — Execução por argumento de linha de comando

O programa deve permitir que o arquivo da matriz seja informado como argumento.

Exemplo:

```
python main.py matriz_caso1.txt ordenacao
python main.py matriz_caso1.txt garcom
```

Também pode ser incluída uma opção para executar todas as soluções:

```
python main.py matriz_caso1.txt todas
```

RF22 — Geração opcional de arquivo de resultados

O sistema pode gerar um arquivo `.txt` ou `.csv` com os resultados obtidos.

Exemplo:

```
resultados_ordenacao_caso1.csv
resultados_garcom_caso1.csv
```

Esse requisito é opcional, mas recomendado para facilitar a comparação no relatório.

9. Requisitos Não Funcionais

RNF01 — Linguagem

O sistema deve ser implementado em Python 3.

Recomenda-se utilizar Python 3.10 ou superior.

RNF02 — Clareza do código

O código deve ser organizado, comentado e dividido em módulos.

A implementação deve ser compreensível para apresentação e defesa oral do trabalho.

RNF03 — Modularidade

O sistema deve separar responsabilidades em arquivos diferentes.

Sugestão de estrutura:

```
bar-dos-filosofos/
|
|— main.py
|— grafo.py
|— filosofo.py
|— garrafa.py
|— solucoes.py
|— relatorio.py
|— matrizes/
|   |— caso1.txt
|   |— caso2.txt
|   |— caso3.txt
|— resultados/
|   |— resultado_caso1_ordenacao.csv
|   |— resultado_caso1_garcom.csv
```

RNF04 — Desempenho

A simulação deve executar em tempo aceitável.

O caso 3 deve demorar pouco mais de 2 minutos, conforme esperado no enunciado.

RNF05 — Confiabilidade

O programa não deve travar durante a execução.

A implementação deve evitar deadlocks e garantir que todas as threads finalizem.

RNF06 — Reprodutibilidade

O sistema deve permitir repetir testes com as mesmas matrizes.

Opcionalmente, pode ser usado um valor fixo de semente aleatória para repetir resultados:

```
random.seed(42)
```

RNF07 — Portabilidade

O sistema deve funcionar em diferentes sistemas operacionais, como Windows, Linux e macOS, desde que possuam Python instalado.

RNF08 — Usabilidade

A execução do programa deve ser simples.

O usuário deve conseguir executar a simulação com comandos claros pelo terminal.

RNF09 — Tratamento de erros

O sistema deve exibir mensagens claras quando:

- o arquivo não for encontrado;
- a matriz estiver inválida;
- a solução escolhida não existir;
- houver valores diferentes de 0 ou 1;
- a matriz não for quadrada;

- a matriz não for simétrica.
-

RNF10 — Apresentação dos resultados

Os resultados devem ser exibidos de forma organizada, preferencialmente em tabela.

Os tempos devem ser exibidos com duas casas decimais.

10. Regras de Negócio

RN01 — Cada garrafa pertence a uma aresta

Uma garrafa só existe quando há uma aresta entre dois filósofos.

RN02 — Uma garrafa não pode ser usada por dois filósofos simultaneamente

Toda garrafa deve ter acesso exclusivo.

RN03 — Um filósofo só pode beber se possuir todas as garrafas sorteadas

O filósofo não pode entrar no estado bebendo se tiver adquirido apenas parte das garrafas desejadas.

RN04 — Ao terminar de beber, o filósofo deve liberar todas as garrafas

Nenhuma garrafa pode permanecer bloqueada após o filósofo sair do estado bebendo.

RN05 — Cada filósofo deve cumprir a quantidade definida de ciclos

No caso 1 e no caso 2, cada filósofo deve beber 6 vezes.

No caso 3, cada filósofo deve beber 3 vezes.

RN06 — A simulação só termina quando todos os filósofos completarem seus ciclos

A execução deve aguardar o término de todas as threads.

RN07 — Deadlock torna a solução inválida

Se a simulação travar e algum filósofo nunca conseguir terminar, a solução não deve ser considerada correta.

RN08 — Starvation deve ser avaliada pelos tempos de espera

Mesmo que não ocorra deadlock, a solução precisa ser analisada para verificar se algum filósofo ficou esperando muito mais do que os outros.

11. Casos de Teste Obrigatórios

11.1 Caso 1 — Jantar dos Filósofos Clássico

Matriz com 5 nós em formato circular:

```
0, 1, 0, 0, 1
1, 0, 1, 0, 0
0, 1, 0, 1, 0
0, 0, 1, 0, 1
1, 0, 0, 1, 0
```

Cada filósofo deve beber 6 vezes.

11.2 Caso 2 — Bar dos Filósofos com 6 nós e baixa conectividade

Matriz com 6 nós:

```
0, 1, 0, 0, 0, 1
1, 0, 1, 0, 0, 0
0, 1, 0, 1, 0, 1
0, 0, 1, 0, 1, 1
0, 0, 0, 1, 0, 1
1, 0, 1, 1, 1, 0
```

Cada filósofo deve beber 6 vezes.

11.3 Caso 3 — Bar dos Filósofos com 12 nós e alta conectividade

Matriz com 12 nós:

```
0, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0
1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0
1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0
0, 1, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0
1, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0
0, 0, 0, 1, 0, 0, 1, 1, 0, 0, 0, 0
0, 0, 0, 1, 1, 1, 0, 1, 0, 1, 0, 0
0, 0, 0, 0, 0, 1, 1, 0, 1, 1, 0, 0
0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 1
0, 0, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1
0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1
0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0
```

Cada filósofo deve beber 3 vezes.

12. Soluções a Implementar

12.1 Solução A — Ordenação de Recursos

Essa solução consiste em numerar todas as garrafas e obrigar que os filósofos sempre adquiram os recursos em uma ordem global fixa.

Funcionamento

1. Cada garrafa recebe um ID único.
2. O filósofo sorteia as garrafas desejadas.
3. Antes de tentar adquirir as garrafas, ele ordena a lista pelo ID.
4. O filósofo tenta adquirir as garrafas seguindo a ordem crescente.
5. Após conseguir todas, entra no estado bebendo.
6. Ao terminar, libera todas as garrafas.

Vantagem

Reduz o risco de deadlock, pois elimina a espera circular.

Possível desvantagem

Pode causar maior tempo de espera para alguns filósofos, dependendo da estrutura do grafo e da ordem dos recursos.

12.2 Solução B — Garçom

Essa solução adiciona uma entidade central chamada garçom, responsável por controlar quantos filósofos podem tentar pegar garrafas ao mesmo tempo.

Funcionamento

1. O filósofo fica com sede.
2. Antes de tentar pegar qualquer garrafa, solicita permissão ao garçom.
3. O garçom permite no máximo $N - 1$ filósofos tentando pegar garrafas simultaneamente.
4. O filósofo tenta adquirir as garrafas.
5. Após beber e liberar as garrafas, libera também a permissão do garçom.

Vantagem

É simples de implementar e reduz o risco de deadlock.

Possível desvantagem

Pode diminuir o paralelismo, pois limita a quantidade de filósofos tentando beber ao mesmo tempo.

13. Métricas de Avaliação

O sistema deverá coletar as seguintes métricas:

13.1 Tempo total da simulação

Tempo desde o início da execução até o término de todos os filósofos.

13.2 Tempo total tranquilo por filósofo

Soma de todos os períodos em que o filósofo ficou tranquilo.

13.3 Tempo total com sede por filósofo

Soma de todos os períodos em que o filósofo ficou esperando para conseguir as garrafas.

13.4 Tempo total bebendo por filósofo

Soma dos períodos em que o filósofo ficou bebendo.

Como cada bebida dura 1 segundo, esse valor deve ser aproximadamente igual ao número de vezes que o filósofo bebeu.

13.5 Tempo médio de espera

Tempo médio que cada filósofo ficou com sede antes de conseguir beber.

13.6 Diferença entre maior e menor espera média

Essa métrica ajuda a avaliar se houve desequilíbrio entre os filósofos.

13.7 Ocorrência de deadlock

A simulação deve verificar se todas as threads terminaram corretamente.

13.8 Indício de starvation

Se um filósofo apresentar tempo médio de espera muito maior do que os demais, pode haver indício de starvation.

14. Critérios de Aceitação

O trabalho será considerado adequado se atender aos seguintes critérios:

1. O programa executa sem erros.
2. O programa lê corretamente arquivos `.txt` com matrizes de adjacência.
3. Os filósofos são executados concorrentemente.
4. As garrafas são acessadas com exclusão mútua.
5. Nenhuma garrafa é usada por dois filósofos ao mesmo tempo.
6. O sistema implementa pelo menos duas soluções.
7. As soluções não apresentam deadlock.
8. Os tempos dos estados são registrados corretamente.
9. O tempo médio de espera é calculado para cada filósofo.
10. Os resultados das soluções são comparados.
11. A saída final é clara e organizada.
12. O código está bem estruturado e compreensível.
13. O relatório explica o funcionamento do problema, das soluções e dos resultados.

15. Sugestão de Estrutura de Código

15.1 Arquivo `main.py`

Responsável por:

- receber argumentos do terminal;
- carregar a matriz;
- escolher a solução;
- iniciar a simulação;
- exibir os resultados.

15.2 Arquivo **grafo.py**

Responsável por:

- ler a matriz de adjacência;
- validar a matriz;
- identificar os vértices;
- identificar as arestas;
- criar a relação entre filósofos e garrafas.

15.3 Arquivo **garrafa.py**

Responsável por representar a garrafa.

A classe **Garrafa** pode conter:

- identificador;
- filósofos conectados;
- **Lock** associado.

15.4 Arquivo **filosofo.py**

Responsável pela classe **Filosofo**.

A classe deve herdar ou usar **threading.Thread**.

15.5 Arquivo **solucoes.py**

Responsável pelas estratégias de sincronização.

Pode conter:

- solução por ordenação de recursos;
- solução do garçom;
- funções auxiliares para aquisição e liberação de garrafas.

15.6 Arquivo **relatorio.py**

Responsável por:

- montar tabela de resultados;
- calcular médias;

- comparar soluções;
 - salvar os resultados em `.csv` ou `.txt`.
-

16. Exemplo de Execução Esperada

Executar solução por ordenação:

```
python main.py matrizes/caso1.txt ordenacao
```

Executar solução do garçom:

```
python main.py matrizes/caso1.txt garcom
```

Executar todas as soluções:

```
python main.py matrizes/caso1.txt todas
```

17. Saída Esperada

O programa deve apresentar uma saída semelhante a esta:

```
=====
Simulação do Problema do Bar dos Filósofos
Solução utilizada: Ordenação de Recursos
Arquivo de entrada: caso1.txt
Quantidade de filósofos: 5
Quantidade de garrafas: 5
Ciclos por filósofo: 6
=====
```

```
Filósofo 0 terminou.
Filósofo 1 terminou.
Filósofo 2 terminou.
Filósofo 3 terminou.
Filósofo 4 terminou.
```

Resultados:

Filósofo	Tranquilo	Com sede	Bebendo	Bebeu	Espera média
F0	12.34s	2.10s	6.00s	6	0.35s
F1	10.98s	3.42s	6.00s	6	0.57s
F2	11.21s	2.89s	6.00s	6	0.48s

F3	13.05s	1.95s	6.00s	6	0.32s
F4	12.77s	2.50s	6.00s	6	0.41s

Tempo total da simulação: 25.80s

Maior espera média: 0.57s

Menor espera média: 0.32s

Diferença: 0.25s

Deadlock detectado: Não

Indício de starvation: Não

18. Testes Recomendados

Teste 1 — Leitura de matriz válida

Verificar se o sistema consegue ler corretamente os três casos fornecidos.

Teste 2 — Matriz inválida

Testar uma matriz não quadrada e verificar se o sistema exibe erro.

Teste 3 — Matriz não simétrica

Testar uma matriz em que $M[i][j]$ seja diferente de $M[j][i]$.

Teste 4 — Execução com solução por ordenação

Executar os três casos usando a solução por ordenação de recursos.

Teste 5 — Execução com solução do garçom

Executar os três casos usando a solução do garçom.

Teste 6 — Comparação entre soluções

Comparar o tempo total e o tempo médio de espera das duas soluções para o mesmo grafo.

Teste 7 — Verificação de deadlock

Confirmar se todos os filósofos terminam a execução.

Teste 8 — Verificação de starvation

Confirmar se nenhum filósofo apresenta espera extremamente superior aos demais.

19. Riscos do Projeto

Risco 1 — Deadlock

Pode ocorrer caso os filósofos segurem parte das garrafas e fiquem esperando indefinidamente por outras.

Mitigação: usar ordenação de recursos ou controle por garçom.

Risco 2 — Starvation

Pode ocorrer se um filósofo ficar esperando por muito mais tempo que os demais.

Mitigação: analisar tempos médios de espera e repetir testes.

Risco 3 — Erro na leitura da matriz

Uma matriz inválida pode comprometer a simulação.

Mitigação: validar a matriz antes de iniciar as threads.

Risco 4 — Liberação incorreta de locks

Se uma garrafa não for liberada, o programa pode travar.

Mitigação: usar blocos `try/finally`.

Risco 5 — Resultados muito variáveis

Como há aleatoriedade, os resultados podem variar entre execuções.

Mitigação: usar uma semente fixa em alguns testes.

20. Entregáveis do Trabalho

O grupo deverá entregar:

1. Código-fonte em Python;
2. Arquivos `.txt` com as matrizes dos três casos;
3. Arquivo README com instruções de execução;
4. Resultados das simulações;
5. Relatório explicando:
 - o problema;
 - a modelagem por grafo;
 - as soluções implementadas;
 - os mecanismos de sincronização usados;

- a comparação entre as soluções;
 - análise de deadlock;
 - análise de starvation;
 - conclusão.
-

21. Sugestão de Divisão do Trabalho em Grupo

Como o trabalho deve ter entre 2 e 4 alunos, uma divisão possível é:

Aluno 1

- Implementar leitura e validação da matriz;
- Criar estrutura do grafo;
- Criar classe Garrafa.

Aluno 2

- Implementar classe Filósofo;
- Controlar estados e tempos;
- Registrar métricas individuais.

Aluno 3

- Implementar solução por ordenação de recursos;
- Testar nos três casos.

Aluno 4

- Implementar solução do garçom;
- Criar comparação dos resultados;
- Auxiliar no relatório final.

Caso o grupo tenha apenas 2 alunos, cada aluno pode ficar responsável por uma solução, e ambos dividem a estrutura base e o relatório.

22. Sugestão de Cronograma

Etapa 1 — Entendimento do problema

- Ler o enunciado;
- Entender a diferença entre Jantar dos Filósofos e Bar dos Filósofos;
- Definir as soluções que serão implementadas.

Etapa 2 — Estrutura inicial

- Criar arquivos do projeto;
- Implementar leitura da matriz;
- Criar classes básicas.

Etapa 3 — Implementação da primeira solução

- Implementar ordenação de recursos;
- Testar com o caso 1;
- Corrigir possíveis erros.

Etapa 4 — Implementação da segunda solução

- Implementar solução do garçom;
- Testar com o caso 1;
- Comparar com a primeira solução.

Etapa 5 — Testes completos

- Executar casos 1, 2 e 3;
- Coletar tempos;
- Verificar deadlock e starvation.

Etapa 6 — Relatório final

- Explicar o problema;
- Descrever a implementação;
- Apresentar tabelas;
- Comparar resultados;
- Escrever conclusão.

23. Conclusão dos Requisitos

O sistema proposto deverá implementar uma simulação concorrente do Problema do Bar dos Filósofos utilizando Python. A implementação deverá representar filósofos como threads e garrafas como recursos compartilhados protegidos por mecanismos de exclusão mútua.

O programa deverá ser capaz de ler diferentes grafos por meio de matrizes de adjacência, executar pelo menos duas soluções de sincronização, registrar os tempos de cada filósofo e comparar os resultados obtidos.

A principal preocupação do projeto será garantir que não ocorra deadlock e que os tempos de espera dos filósofos sejam equilibrados, evitando starvation. Dessa forma, o trabalho permitirá demonstrar, na prática, conceitos importantes de programação concorrente,

sincronização de processos, exclusão mútua e análise de desempenho em sistemas computacionais.

24. Diferenciais Recomendados

Além dos requisitos obrigatórios definidos para a implementação do Problema do Bar dos Filósofos, o projeto poderá incluir alguns diferenciais com o objetivo de tornar a solução mais completa, organizada e tecnicamente mais interessante. Esses diferenciais não substituem os requisitos principais, mas agregam valor ao trabalho, principalmente na análise dos resultados e na apresentação final.

24.1 Modo de Comparação Automática entre Soluções

O sistema poderá possuir um modo de execução responsável por rodar automaticamente os casos de teste com todas as soluções implementadas.

Em vez de executar manualmente cada combinação de caso e solução, o programa poderá executar, de forma sequencial:

- Caso 1 com Ordenação de Recursos;
- Caso 1 com Garçom;
- Caso 2 com Ordenação de Recursos;
- Caso 2 com Garçom;
- Caso 3 com Ordenação de Recursos;
- Caso 3 com Garçom.

Ao final, o sistema deverá apresentar uma tabela comparativa contendo informações como tempo total de execução, espera média geral, maior tempo médio de espera, menor tempo médio de espera e possível indício de starvation.

Esse diferencial torna o trabalho mais completo, pois facilita a comparação entre as estratégias de sincronização e demonstra que o grupo analisou o desempenho das soluções de forma organizada.

Exemplo de comando:

```
python main.py --comparar
```

24.2 Execução de Múltiplos Experimentos

Como a simulação utiliza valores aleatórios, como o tempo no estado tranquilo e a escolha das garrafas, os resultados podem variar entre uma execução e outra. Por isso, o sistema poderá permitir a execução de várias rodadas para o mesmo caso e a mesma solução.

Por exemplo, o programa poderá executar cada cenário cinco vezes e calcular a média dos resultados obtidos.

Métricas que podem ser calculadas:

- tempo médio total da simulação;
- tempo médio de espera dos filósofos;
- maior espera média registrada;
- menor espera média registrada;
- diferença média entre filósofos;
- melhor e pior execução.

Esse diferencial melhora a confiabilidade da análise, pois evita que a conclusão do trabalho seja baseada em apenas uma execução isolada.

Exemplo de comando:

```
python main.py matrizes/caso1.txt ordenacao --experimentos 5
```

24.3 Geração de Arquivos CSV com os Resultados

O sistema poderá salvar os resultados das simulações em arquivos `.csv`, permitindo que os dados sejam abertos em ferramentas como Excel, Google Planilhas ou LibreOffice Calc.

Cada arquivo poderá conter os dados individuais de cada filósofo e também os dados gerais da simulação.

Colunas sugeridas:

caso, solucao, filosofo, tempo_tranquilo, tempo_com_sede, tempo_bebendo, vezes_bebeu, espera_media

Também poderá ser gerado um arquivo comparativo final:

comparativo_final.csv

Esse diferencial facilita a organização dos dados, a construção de tabelas no relatório e a conferência dos resultados pelo professor.

24.4 Geração de Gráficos Simples

A partir dos resultados obtidos, o sistema poderá gerar gráficos simples para facilitar a interpretação dos dados.

Gráficos recomendados:

- tempo médio de espera por filósofo;
- comparação do tempo total entre soluções;
- comparação da espera média geral entre soluções;
- diferença entre maior e menor espera média;
- comparação entre os casos 1, 2 e 3.

Os gráficos podem ser gerados usando a biblioteca `matplotlib`.

Esse diferencial valoriza bastante a apresentação, pois permite visualizar de forma clara se uma solução foi mais eficiente ou mais equilibrada que outra.

Exemplos de arquivos gerados:

grafico_espera_caso1_ordenacao.png

grafico_comparativo_caso1.png

grafico_tempo_total_solucoes.png

24.5 Índice de Justiça para Avaliação de Starvation

Além de mostrar o tempo médio de espera de cada filósofo, o sistema poderá calcular um índice simples de justiça para avaliar se houve desequilíbrio entre os filósofos.

Uma forma simples de calcular esse índice é comparar a maior espera média com a menor espera média.

Exemplo:

$\text{índice de desequilíbrio} = \text{maior espera média} / \text{menor espera média}$

Outra possibilidade é calcular a diferença absoluta:

$\text{diferença de espera} = \text{maior espera média} - \text{menor espera média}$

Com isso, o sistema poderá apresentar uma análise automática como:

Maior espera média: 1.20s

Menor espera média: 0.40s

Índice de desequilíbrio: 3.0

Conclusão: há diferença relevante de espera, mas todos os filósofos conseguiram beber.

Esse diferencial contribui diretamente para a avaliação de starvation, pois torna a análise mais objetiva e mensurável.

24.6 Monitor de Deadlock

O sistema poderá possuir um monitor de execução, também chamado de watchdog, responsável por verificar se a simulação ficou muito tempo sem progresso.

A ideia é registrar o momento em que algum filósofo conseguiu beber pela última vez. Se nenhum filósofo conseguir beber após um determinado intervalo de tempo, o programa poderá exibir um alerta.

Exemplo:

[MONITOR] Nenhum filósofo bebe há 15 segundos. Possível deadlock ou espera excessiva.

Caso todos os filósofos terminem normalmente, o sistema poderá exibir:

Status final: execução concluída sem deadlock.

Esse diferencial é interessante porque mostra que o grupo não apenas implementou soluções para evitar deadlock, mas também criou uma forma prática de observar a ocorrência ou não desse problema durante a execução.

24.7 Modo Verbose com Log da Simulação

O sistema poderá oferecer um modo detalhado de execução, ativado por argumento de linha de comando.

Nesse modo, o programa deverá mostrar os principais eventos da simulação, como:

- filósofo ficou tranquilo;
- filósofo ficou com sede;
- filósofo sorteou determinadas garrafas;
- filósofo conseguiu pegar uma garrafa;

- filósofo começou a beber;
- filósofo liberou as garrafas;
- filósofo terminou seus ciclos.

Exemplo de saída:

F2 ficou com sede.

F2 deseja as garrafas G1 e G3.

F2 pegou G1.

F2 pegou G3.

F2 está bebendo.

F2 liberou G1 e G3.

F2 voltou ao estado tranquilo.

Exemplo de comando:

```
python main.py matrizes/caso1.txt ordenacao --verbose
```

Esse diferencial ajuda na apresentação do trabalho, pois permite demonstrar visualmente o comportamento das threads e o controle de acesso aos recursos compartilhados.

24.8 Semente Aleatória Configurável

O sistema poderá permitir que o usuário informe uma semente aleatória para controlar os sorteios feitos durante a simulação.

Isso permite repetir uma execução com os mesmos valores aleatórios, facilitando testes, comparação e depuração.

Exemplo de comando:

```
python main.py matrizes/caso1.txt ordenacao --seed 42
```

Com esse recurso, o grupo poderá repetir uma mesma execução durante a apresentação, caso seja necessário demonstrar determinado comportamento.

Esse diferencial melhora a reprodutibilidade dos experimentos e transmite maior organização técnica.

24.9 Relatório Automático de Execução

O sistema poderá gerar automaticamente um relatório em formato `.txt`, contendo um resumo da simulação executada.

Esse relatório poderá incluir:

- nome do arquivo de entrada;
- matriz utilizada;
- quantidade de filósofos;
- quantidade de garrafas;
- solução utilizada;
- quantidade de ciclos por filósofo;
- tabela de tempos;
- tempo total da simulação;
- tempo médio de espera;
- análise de deadlock;
- análise de starvation;
- conclusão automática.

Exemplo de conclusão automática:

A solução Ordenação de Recursos finalizou sem ocorrência de deadlock.

Todos os filósofos completaram a quantidade definida de ciclos.

A diferença entre a maior e a menor espera média foi de 0.35s.

Não há forte indício de starvation nesta execução.

Esse diferencial facilita a elaboração do relatório final do trabalho e demonstra cuidado com documentação e análise dos resultados.

24.10 Implementação de uma Terceira Solução: Randomized Backoff

Caso as duas soluções principais já estejam funcionando corretamente, o grupo poderá implementar uma terceira solução baseada em Randomized Backoff.

Nessa estratégia, quando um filósofo não consegue adquirir todas as garrafas necessárias, ele libera as garrafas que já conseguiu pegar, espera um tempo aleatório e tenta novamente.

Funcionamento básico:

1. O filósofo fica com sede.
2. Sorteia as garrafas desejadas.
3. Tenta adquirir as garrafas.
4. Se não conseguir todas, libera as que já pegou.
5. Aguarda um tempo aleatório.
6. Tenta novamente.
7. Ao conseguir todas, entra no estado bebendo.
8. Após beber, libera todas as garrafas.

Essa solução é mais simples que Chandy-Misra e ainda permite uma comparação interessante com as soluções de Ordenação de Recursos e Garçom.

Esse diferencial deve ser considerado opcional, pois só deve ser implementado se a estrutura principal do projeto já estiver estável.

25. Priorização dos Diferenciais

Para evitar aumento excessivo da complexidade do projeto, recomenda-se priorizar os diferenciais na seguinte ordem:

1. Modo de comparação automática entre soluções;
2. Geração de arquivos CSV com os resultados;
3. Índice de justiça para avaliação de starvation;
4. Semente aleatória configurável;
5. Modo verbose com log da simulação;
6. Geração de gráficos simples;
7. Monitor de deadlock;
8. Relatório automático de execução;
9. Terceira solução com Randomized Backoff.

Os diferenciais mais recomendados para implementação são o modo de comparação automática, a geração de CSV, o índice de justiça e a semente configurável, pois possuem baixa complexidade e agregam bastante valor à análise do trabalho.

Caso haja tempo disponível, a geração de gráficos e o relatório automático também podem enriquecer bastante a entrega final. A terceira solução deve ser tratada como um adicional mais avançado, sendo implementada somente depois que as duas soluções obrigatórias estiverem funcionando corretamente.

26. Plano de Desenvolvimento

Para facilitar a implementação do projeto, recomenda-se seguir uma ordem progressiva de desenvolvimento, começando pelas partes básicas do sistema e avançando gradualmente para as soluções concorrentes e os diferenciais.

A implementação deve ser feita inicialmente com o Caso 1, referente ao Jantar dos Filósofos clássico, pois ele possui menor quantidade de nós e uma estrutura mais simples para testes. Após a validação do funcionamento básico, o sistema deverá ser testado nos Casos 2 e 3.

26.1 Etapa 1 — Estrutura Inicial do Projeto

Criar a estrutura de pastas e arquivos do projeto, separando os módulos principais, como leitura do grafo, classe dos filósofos, classe das garrafas, soluções de sincronização e geração de resultados.

26.2 Etapa 2 — Criação dos Arquivos de Entrada

Criar os arquivos `.txt` contendo as matrizes de adjacência dos três casos indicados no enunciado do trabalho.

26.3 Etapa 3 — Leitura e Validação da Matriz

Implementar a leitura da matriz de adjacência a partir do arquivo `.txt`. Em seguida, validar se a matriz é quadrada, simétrica, possui apenas valores 0 e 1 e apresenta diagonal principal igual a zero.

26.4 Etapa 4 — Criação das Garrafas

A partir da matriz de adjacência, identificar as arestas do grafo e criar uma garrafa para cada aresta. Cada garrafa deverá possuir um identificador único e um mecanismo de exclusão mútua.

26.5 Etapa 5 — Criação da Classe Filósofo

Implementar a classe `Filosofo`, contendo identificador, lista de garrafas adjacentes, estado atual, contadores de tempo e quantidade de vezes que o filósofo bebeu.

26.6 Etapa 6 — Simulação com Threads

Transformar cada filósofo em uma thread, permitindo que os filósofos executem de forma concorrente. Cada filósofo deverá alternar entre os estados tranquilo, com sede e bebendo.

26.7 Etapa 7 — Implementação da Primeira Solução

Implementar inicialmente a solução por Ordenação de Recursos, pois ela é mais simples e reduz a possibilidade de deadlock ao obrigar que as garrafas sejam adquiridas sempre em uma ordem global fixa.

26.8 Etapa 8 — Registro dos Tempos

Registrar os tempos de cada filósofo nos estados tranquilo, com sede e bebendo. Também deverão ser calculados o tempo médio de espera, a quantidade de vezes que cada filósofo bebeu e o tempo total da simulação.

26.9 Etapa 9 — Exibição dos Resultados

Criar uma saída organizada em formato de tabela, exibindo os tempos individuais de cada filósofo e os dados gerais da simulação.

26.10 Etapa 10 — Implementação da Segunda Solução

Após a primeira solução estar funcionando corretamente, implementar a solução do Garçom, utilizando um mecanismo de controle para limitar a quantidade de filósofos tentando adquirir garrafas ao mesmo tempo.

26.11 Etapa 11 — Comparação entre as Soluções

Executar as duas soluções nos casos de teste e comparar os resultados obtidos. A comparação deverá considerar tempo total da simulação, tempo médio de espera, maior e menor espera média, indícios de starvation e ocorrência ou não de deadlock.

26.12 Etapa 12 — Implementação dos Diferenciais

Somente após a implementação dos requisitos principais, deverão ser adicionados os diferenciais do projeto, como geração de CSV, semente aleatória configurável, modo verbose, gráficos, índice de justiça e comparação automática entre soluções.

26.13 Ordem Resumida de Implementação

A ordem recomendada para o desenvolvimento é:

1. Criar a estrutura do projeto;
2. Criar os arquivos das matrizes;
3. Implementar leitura e validação da matriz;
4. Criar as garrafas a partir das arestas;
5. Criar a classe Filósofo;
6. Executar filósofos como threads;
7. Implementar Ordenação de Recursos;
8. Registrar tempos;
9. Mostrar resultados em tabela;
10. Implementar solução do Garçom;
11. Comparar as soluções;
12. Adicionar diferenciais;
13. Finalizar o relatório.

Essa ordem reduz o risco de erros, pois permite testar cada parte do sistema separadamente antes de avançar para etapas mais complexas.

